

Dilema do Agente Duplo

- **Limite de tempo:** 12 minutos.
- **Armazenamento:** 5 GB
- **Ambiente:** uma GPU (≈ 16 GB VRAM), sem acesso à Internet
- **Tamanho da solução:** `solution.ipynb` ≤ 1 MB
- **Pontuação de referência:** 0

No centro nacional de IA em Astana, dois modelos computacionais — o Modelo R (uma ResNet-18) e o Modelo V (um ViT-Tiny) — estão a analisar fotografias. Neste momento, ambos os modelos estão a fazer um trabalho perfeito, obtendo uma precisão de 100% e concordando em todas as imagens. Para testar até que ponto os seus «cérebros» inteligentes são realmente diferentes, o cientista encarregue do centro de IA lança o seguinte desafio: fazer alterações minúsculas, quase invisíveis, aos píxeis de cada fotografia, de modo que o Modelo R e o Modelo V discordem completamente.



1. Tarefa

Dois classificadores de imagens pré-treinados observam a mesma imagem. Nas imagens fornecidas nesta tarefa, ambos os classificadores inicialmente têm uma precisão de 100%.

- **Modelo R:** `torchvision.models.resnet18` (uma CNN, ResNet18).
- **Modelo V:** `vit_tiny_patch16_224` da `timm` (um Transformer, ViT-Tiny).

A sua tarefa consiste em criar uma pequena alteração («perturbação») para cada imagem, de modo que os dois modelos discordem. Para cada imagem, tem de criar **duas perturbações diferentes**:

- **Tipo A:** depois de a adicionar, o Modelo R continua a classificar corretamente a imagem, mas o Modelo V classifica-a incorretamente.
- **Tipo B:** depois de a adicionar, o Modelo V continua a classificar corretamente a imagem, mas o Modelo R classifica-a incorretamente.

Cada perturbação tem de ser suficientemente *pequena* para que seja difícil de detetar. Perturbações menores obtêm pontuações mais elevadas (consulte a Secção 5). A perturbação é aplicada diretamente à imagem original no nível dos píxeis.

2. Dados públicos

É fornecido com a tarefa um conjunto de imagens, organizado em duas partições — `train` (100 imagens) e `test_public` (100 imagens) — cada uma destas partições com imagens de resolução variável. Todas as imagens pertencem às 1000 classes do ImageNet-1K, e tanto o Modelo R como o Modelo V obtêm uma precisão de 100% em ambas as partições.

São fornecidos os seguintes ficheiros:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

Durante a avaliação, a sua pasta `dataset/test_public/` é substituída de forma transparente por um de dois conjuntos ocultos de imagens (`test_leaderboard_a` e `test_leaderboard_b`) usadas para calcular as pontuações oficiais. Cada uma contém **100 imagens** em formato PNG e um ficheiro de etiquetas.

Nota: Para esta tarefa, as etiquetas dos datasets de teste estão acessíveis.

3. Formato da saída

Para cada imagem, tem de produzir dois ficheiros:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- `{index}` (0, 1, 2, ...), corresponde ao nome da imagem nos datasets.
- Cada ficheiro é apenas um único tensor guardado com `torch.save`. A sua forma tem de ser `3 x H x W`, em que `H` e `W` correspondem à resolução **original** dessa imagem (e não `224 x 224`).
- O código deve produzir apenas um ficheiro ZIP, `submission.zip`. Coloque todos os ficheiros `.pt` no nível superior do arquivo ZIP, sem qualquer pasta envolvente ou subdiretórios.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

O notebook irá alertá-lo se existir quaisquer problemas com o formato da saída.

4. Restrições

- **Modelos:** Tem de utilizar `torchvision.models.resnet18(pretrained=True)` e `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Não são permitidos outros modelos pré-treinados.
- **Pipeline de transformações (imposto durante a avaliação):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` para obter mais detalhes.
- **Resolução da perturbação:** Tem de corresponder à resolução **original** da imagem (não `224x224`). O tensor é adicionado à imagem em bruto *antes* do pipeline de transformações.
- **Formato da saída:** apenas ficheiros `.pt` — sem PNG/JPG. Os tensores são adicionados à imagem em bruto e os valores dos píxeis são limitados a `[0, 1]` antes do pré-processamento.
- **Nomenclatura dos ficheiros:** Listagem plana, no formato estrito `{index}_a.pt` / `{index}_b.pt`. Sem subdiretórios dentro do zip.
- **Bibliotecas:** `torch`, `torchvision`, `timm`.

5. Pontuação

A pontuação final é calculada da seguinte forma. Seja M o número de imagens na partição, $Score_A$ o número de perturbações de Tipo A bem-sucedidas e $Score_B$ o número de perturbações de Tipo B bem-sucedidas:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF é uma função concebida para penalizar perturbações com uma norma elevada e para ser muito sensível perto do limite máximo de desempenho. Ela está limitada ao intervalo de 0.5 a 1. A

implementação completa pode ser consultada na Secção 8 de `solution.ipynb`.

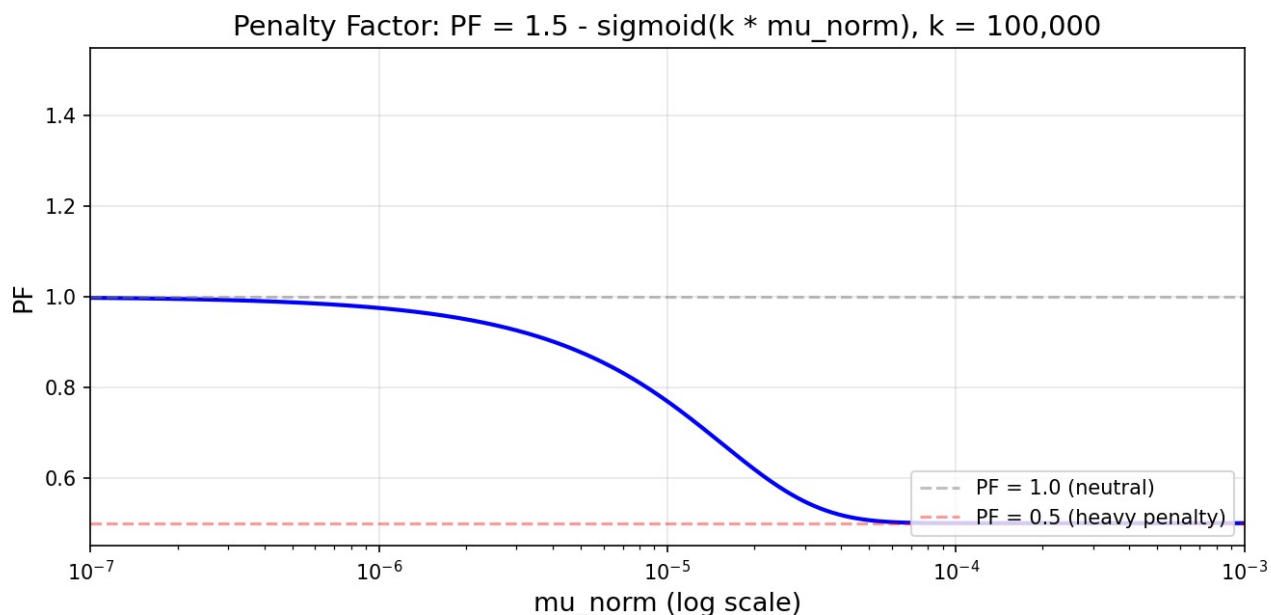


Figura: A curva da função de penalização.

6. Verificar a submissão

Existem verificações no notebook que o alertam caso haja problemas de formatação, na Secção 7 do notebook `solution.ipynb`.

7. Testes locais

`solution.ipynb` contém um exemplo completo e funcional. Carrega os dados públicos, ambos os modelos e o avaliador oficial, e escreve um ficheiro ZIP de submissão. Leia-o antes de começar.

8. Como submeter

- Guarde as suas alterações em `solution.ipynb`.
- Abra o separador Git na barra lateral esquerda do JupyterLab.
- Faça **Stage** de `solution.ipynb` (o ícone + junto ao ficheiro).
- Introduza uma mensagem de commit e clique em **Commit**.
- Clique no ícone da nuvem com uma seta para cima para fazer push.
- Regresse a esta página do concurso e clique em **Submit**.

Submeta exatamente um ficheiro, denominado `solution.ipynb`.